CS3211 Concurrent Programming

AY 25/26 Sem 2 — github/omgeta

1. Introduction

Concurrency refers to a system performing multiple independent tasks in overlapping time periods.

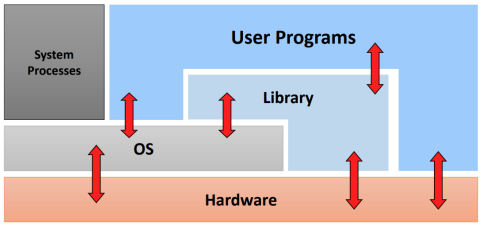
Concurrency vs. Parallelism:

i. Concurrency: tasks progress in overlapping time
ii. Parallelism: tasks also execute simultaneously
   - Enabled by multiple cores, each can have multiple hardware threads (SMT)
   - Hardware threads dictate parallel execution of kernel threads

Program Parallelization Steps:

i. Decomposition: split problem into parallel tasks
   - Granularity: size of task
ii. Scheduling: assign tasks to threads
iii. Orchestration: synchronisation and communication of tasks to satisfy dependencies
iv. Mapping: bind threads to physical cores

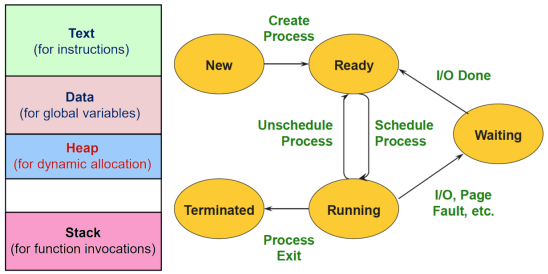
Software Stack:



Processes

Process is an abstraction for a running program:

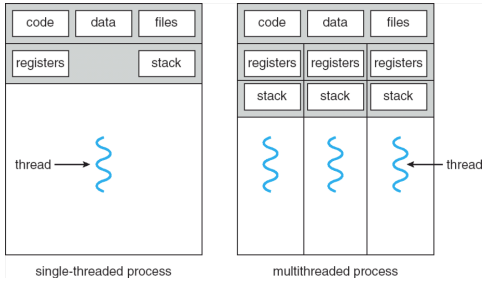
– High system call overhead
– OS allocates and initializes data structures
– Communication costly through the OS



Threads

Threads are independent execution flows within a process:

i. Shared: Resources, Address Space
ii. Private: Thread ID, Registers, "Stack" (diff. SP)
+ Efficient: cheaper creation and context switching
+ Resource Sharing: process resources can be shared



Implementations:

i. User-Threads are managed by a userspace library:
   + Fast: context switches have low overhead
   – Cannot parallelize to different cores
   – Threads blocking will block whole process
ii. Kernel-Threads are implemented in the OS:
   + Parallel: threads can run across many cores
   – Slower: thread operations are system calls

Synchronisation

Race Condition:

i. Program outcome depends on timing/ordering of concurrent operations

Data Races lead to undefined behaviour:

i. Two concurrent access to same memory location
ii. At least one access is a write
iii. No synchronization

Mutual Exclusion synchronises access to shared resources:

i. Thread locks mutex to access shared resources
ii. Same thread unlocks mutex once shared resources no longer used

Symptoms of Incorrect Synchronization:

i. Deadlock: there is an execution where no progress is made when it should be possible
   - Indistinguishable from livelock
ii. Starvation: a thread cannot progress because other threads hold a required resource, which it does not get a chance to acquire

Mechanisms:

i. Lock: primitive `acquire()`, `release()`
ii. Semaphore: atomic ≥ 0; `acquire()`, `release()`
   - Binary: represents single access
   - Counting: number of threads in CS $\leq N$
   - Can be manipulated by different threads
   - No relation to data being controlled
iii. Monitors: high-level implicit concurrency
iv. Messages: explicit model of communication and synchronisation based on atomic data transfer

2. C++

Threads

Operations:

i. Construct:
   - Function: `std::thread t(f, args...)`
   - Callable object: `std::thread t{F{}, args...}`
   - Lambda: `std::thread t([]{ ... })`

ii. Wait Once: `t.join()`
   - Check: `t.joinable()`
   - Must still join on exception paths

iii. Detach: `t.detach()`
   – Detached thread no longer joinable
   – May outlive local variable references

iv. Pass By Reference: `std::ref(x)`

v. Ownership: movable, not copyable

Mutexes

Operations:

i. Construct: `std::mutex m`

ii. Explicit: `m.lock()`, `m.unlock()`
   – Must unlock on every exit path, incl. exceptions

iii. RAII: `std::lock_guard<std::mutex> g(m)`
   + Locks on construction and unlocks on scope exit

iv. Flexible: `std::unique_lock<std::mutex> lk(m)`
   - Adopt Existing Lock: `lk(m, std::adopt_lock)`
   - Defer Lock: `lk(m, std::defer_lock)`
   + Unlocks on scope exit if mutex owned
   + Allows multiple lock/unlock within a scope
   + Needed for condition variables
   + Moveable: can transfer lock ownership

v. Multiple Mutexes (Deadlock-Free):
   - Lock in total order, avoid nested locks
   - Explicit: `std::lock(m1, m2, ...)`
   - RAII: `std::scoped_lock lk(m1, m2, ...)`

Condition Variables

Condition variables sync actions without busy-waiting.

Operations:

i. Construct: `std::condition_variable cv`
   - Works only on `std::unique_lock<std::mutex>`
   - Generic (any mutex-like, less efficient): `std::condition_variable_any cva`

ii. Wait: `cv.wait(lk)` / `cv.wait(lk, pred)`
   - Checks predicate with mutex locked
   - If condition is met, returns
   - If condition is unmet, unlocks mutex and blocks
   – Spurious Wake: may wake without notification to check condition; so should be side effect-free

iii. Wake One Thread: `cv.notify_one()`

iv. Wake All Threads: `cv.notify_all()`

Monitors

Monitors encapsulate shared data and operations enforced by a mutex and condition variables.

i. One thread executes monitor operations at a time

ii. Threads wait for conditions and state changes

Example:

```
class JobQueue {
  std::mutex m;
  std::condition_variable cv;
  std::queue<Job> q;
public:
  void enqueue(Job j) {
    {
      std::unique_lock<std::mutex> lk(m);
      q.push(j);
    }
    cv.notify_one();
  }

  Job dequeue() {
    std::unique_lock<std::mutex> lk(m);
    cv.wait(lk, [&]{ return !q.empty(); });
    Job j = q.front(); q.pop();
    return x;
  }
};
```

Memory Model

Memory model is the contract about the visible side effects of memory operations.

i. As-if: compiler/hardware may reorder operations if single-threaded observable behavior is unchanged.

Ordering Relations:

i. Sequenced-Before: program order

ii. Synchronizes-With: suitably-tagged store → load on same atomic, if the load reads that value

iii. Happens-Before: transitive closure of Sequenced-Before ∪ Synchronizes-With

iv. Modification Order: per-atomic total order of writes consistent across threads

v. Coherence Order: per-atomic partial order of writes (totally ordered) and reads (unordered w.r.t others) consistent across threads

vi. Visible Side-Effect: write visible to a read

Atomics (`std::atomic<T>`) have indivisible operations:

i. Load: `a.load(std::memory_order)`

ii. Store: `a.store(val, std::memory_order)`

iii. RMW: `a.fetch_add(val, std::memory_order)`, `a.exchange(val, std::memory_order)`

iv. May be lock-free/implemented with mutex: `a.is_lock_free()`, `T::is_always_lock_free`

Memory Models:

i. Sequentially Consistent (`seq_cst`):
   - All threads agree on same order of operations

ii. Relaxed (`relaxed`):
   - No Synchronizes-With cross-thread guarantees

iii. Acquire-Release (`acquire`, `release`, `acq_rel`)
   - Release store → Acquire load on same atomic can establish Synchronizes-With
   - Writes before release become visible after the acquire (in the acquiring thread)
   - Release Sequence: chain of RMWs on the same atomic after release; an acquire reading from the chain synchronizes-with the original release

Lock-based Data Structures

Lock-based structures protect shared state with mutexes.

i. Lock the right mutex before accessing protected data
ii. Hold locks for the minimum time
iii. Interface should not expose race conditions
iv. Deadlocks are possible

Stack (one global mutex)

```
void push(T x) {
  std::lock_guard<std::mutex> lk(m);
  data.push(std::move(x));
}

std::shared_ptr<T> pop() {
  std::lock_guard<std::mutex> lk(m);
  if (data.empty()) throw empty_stack();
  auto res =
    std::make_shared<T>(std::move(data.top()));
  data.pop();
  return res;
}
```

+ Safe for concurrent calls, exception-safe
− Work is serialized
− No built-in waiting for items: must poll `empty()` or catch `empty_stack` exceptions

Queue (one mutex + notification)

```
void push(T x) {
  std::lock_guard<std::mutex> lk(mut);
  data_queue.push(std::move(x));
  data_cond.notify_one();
}

void wait_and_pop(T& value) {
  std::unique_lock<std::mutex> lk(mut);
  data_cond.wait(lk, [this]{ return
    !data_queue.empty(); });
  value = std::move(data_queue.front());
  data_queue.pop();
}
```

+ `wait_and_pop()` blocks until an item is available
+ Safe for concurrent member calls
− One mutex still serializes queue operations
− `notify_one()` + exception in `wait_and_pop()` can leave other waiters asleep

Queue (push shared-pointers in the queue)

```
void push(T x) {
  auto data = std::make_shared<T>(std::move(x));
  std::lock_guard<std::mutex> lk(mut);
  data_queue.push(data);
  data_cond.notify_one();
}

std::shared_ptr<T> wait_and_pop() {
  std::unique_lock<std::mutex> lk(mut);
  data_cond.wait(lk, [this]{ return
    !data_queue.empty(); });
  auto res = data_queue.front();
  data_queue.pop();
  return res;
}
```

+ Data can be allocated before locking
+ Returned value can share ownership with the queue element
− One mutex still serializes queue operations

Queue (fine-grained; dummy node)

```
node* get_back() {
  std::lock_guard<std::mutex> lk(back_mutex);
  return back;
}

std::unique_ptr<node> pop_front() {
  std::lock_guard<std::mutex> lk(front_mutex);
  if (front.get() == get_back()) return nullptr;
  auto old_front = std::move(front);
  front = std::move(old_front->next);
  return old_front;
}

void push(T x) {
  auto new_data =
    std::make_shared<T>(std::move(x));
  auto p = std::make_unique<node>();
  node* new_back = p.get();
  std::lock_guard<std::mutex> lk(back_mutex);
  back->data = new_data;
  back->next = std::move(p);
  back = new_back;
}
```

+ Separate `front_mutex` and `back_mutex`
+ `push()` only locks the back
+ More concurrency than one global mutex
− Locking protocol and empty-check become more complex

Queue (more fine-grained)

```
std::unique_ptr<node> try_pop_head() {
  std::unique_lock<std::mutex>
    front_lk(front_mutex);
  if (front.get() == get_back()) return nullptr;

  std::unique_lock<std::mutex>
    node_lk(front->mutex);
  auto old_head = std::move(front);
  front = std::move(old_head->next);
  return old_head;
}

std::shared_ptr<T> try_pop() {
  std::unique_ptr<node> old_head;
  { old_head = try_pop_head(); }   // lock
    released before delete
  return old_head ? old_head->data :
    std::shared_ptr<T>();
}

void push(T x) {
  auto new_data =
    std::make_shared<T>(std::move(x));
  auto p = std::make_unique<node>();
  node* new_back = p.get();

  std::lock_guard<std::mutex> back_lk(back_mutex);
  std::lock_guard<std::mutex>
    node_lk(back->mutex);
  back->data = new_data;
  back->next = std::move(p);
  back = new_back;
}
```

+ Per-node mutex
+ More fine-grained synchronization between `push()` and `pop()`
– More complex locking protocol
– Must release locks before destroying the old node

Lock-Free Data Structures

Lock-free structures avoid blocking library calls.

i. Obstruction-free: a thread completes if it eventually runs in isolation
ii. Lock-free: from any point, some thread completes
   - Livelock-free but Starvation possible
   + Maximum concurrency: some thread progresses
   + Robust: if one thread dies, others can proceed
   – Atomics may be much slower than non-atomics
   – Contention/write propagation hurt performance
   – Cache ping-pong and false sharing
iii. Wait-free: every running thread completes regardless of execution speed of others
   - Livelock- and Starvation-free; stronger lock-free
   + Per-thread progress guaranteed
   – Harder to design and prove correct
   – Often higher overhead than lock-free structures

Risks:

i. Livelock: from unbounded restarts in response to other thread actions
ii. Starvation: loops with CAS, or algorithms with unbounded number of retries due to clashes

Guidelines:

i. Start with `std::memory_order_seq_cst`
ii. Use CAS loops carefully
iii. Watch for ABA
iv. Use safe memory reclamation
v. Prefer lock-based unless lock-free progress needed

CAS Loop Pattern

```
int fetch_add(std::atomic<int>& value, int add) {
  int old_value = value.load();
  while (true) {
    int new_value = old_value + add;
    if (value.compare_exchange_weak(old_value,
    new_value))
      return old_value;
  }
}
```

Queue (initial idea)

```
void push(Job job) {
  Node* new_dummy = new Node();
  Node* work_node =
    m_queue_back.exchange(new_dummy);
  work_node->job = job;
  work_node->next.store(new_dummy);
}

std::optional<Job> try_pop() {
  Node* old_front = m_queue_front.load();
  while (true) {
    Node* new_front = old_front->next.load();
    if (new_front == QUEUE_END)
      return std::nullopt;
    if (m_queue_front.CAS(old_front, new_front))
      return old_front->job;
  }
}
```

- `exchange()` gives each producer its exclusive dummy node
- `try_pop()` advances the head with a CAS loop

Issue 1: Data races

```
Node* work_node =
    m_queue_back.exchange(new_dummy,
    stdmo::acq_rel);
work_node->next.store(new_dummy, stdmo::release);

Node* old_front =
    m_queue_front.load(stdmo::acquire);
Node* new_front =
    old_front->next.load(stdmo::acquire);
m_queue_front.compare_exchange_weak(old_front,
    new_front, stdmo::acq_rel);
```

- Producer vs. producer: `m_queue_back.exchange(..., stdmo::acq_rel)`
- Producer vs. its consumer: release store to `next`, acquire load from `next`
- Producer vs. next consumer: acquire / acq_rel accesses on `m_queue_front`

Issue 2: ABA

```
struct alignas(16) GenNodePtr {
  Node* node;
  uintptr_t gen;
};

std::atomic<GenNodePtr> m_queue_front;
```

- CAS may succeed even if a pointer changes away from A and back to A
- Fix: add a generation counter to the front pointer

Issue 3: Use-after-free

```
Node* get_recycled_node_or_allocate_new();
void add_node_to_recycling_stack(Node* node);
```

- Old front pointers may become stale
- Fix: do not delete nodes directly; reuse nodes via a lock-free recycling stack

Issue 4: Recycling race

```
Node* cur_stack_next =
  old_stack_top.node->next.load(stdmo::acquire);

node->next.store(old_stack_top.node,
    stdmo::release);
```

- Recycling also needs synchronization
- Use release / acquire on recycled-node `next`

Issue 5: Serializability

- The repaired queue can still be non-serializable: head / tail may be updated before fully linked
- Fix: Michael-Scott Queue prioritizes maintaining a valid linked list

3. Go

Goroutines

Goroutines are lightweight function tasks running in the same address space.

Operations:

i. Start: `go f(args...)`
ii. Lifetime:
   - Program exits when `main` returns
   - Not GC'd: avoid goroutine leaks
iii. Blocks only block the thread, no other goroutines
iv. Preemptable: by Go's runtime

Channels

Channels coordinate goroutines by communication.

Operations:

i. Make: `ch := make(chan T)` / `make(chan T, C)`
   - Directional: `chan<- T` (send-only), `<-chan T` (recv-only)
ii. Send: `ch <- v`
   - Blocks if channel is full (buffered) / until a receiver (unbuffered)
iii. Receive: `v, ok := <-ch`
   - Blocks if channel is empty (can cause deadlocks)
   - `ok` false $\iff$ `v` is default from closed channel
iv. Close: `close(ch)`
   - Allowed from closed channel any number of times
   - Sending to a closed channel panics
v. Range: `for v := range ch { ... }` runs until channel is closed and drained

Ownership Pattern:

i. Owner goroutine makes, writes, and closes channel
ii. Expose only `<-chan T` to consumers

select

`select` composes channel operations:

i. Blocks if no case is ready (unless `default`)
ii. If multiple cases are ready: runtime picks pseudo-randomly (uniform over ready cases)
iii. Stopping:
   - `time.After(d)`: channel that fires after `d`
   - `done` channel to signal stop

For-select Loop:

```
for {
  select {
  case <-done:
    return
  default:
    // do work
  }
}
```

sync Package

Use mostly in small scopes (e.g., inside a `struct`); prefer higher-level coordination with channels when possible.

i. WaitGroup: `var wg sync.WaitGroup`
   - `wg.Add(n)`, `wg.Done()`, `wg.Wait()`, `wg.Go(f)`
ii. Mutex: `var mu sync.Mutex`:
   - Locks: `mu.Lock()`, `mu.Unlock()`
iii. Read-Write Mutex: `var rw sync.Mutex`:
   - Reads: `rw.RLock()`, `rw.RUnlock()`
   - Writes: `rw.Lock()`, `rw.Unlock()`
iv. Cond: `c := sync.NewCond(&mu)`
   - Wait: `c.Wait()` unlocks & blocks, then re-locks before returning
   - Wake: `c.Signal()` / `c.Broadcast()`
v. Once: `once.Do(f)` runs `f` at most once
vi. Pool: `p.Get()` / `p.Put(x)`

Memory Model

Go's memory model defines when a read in one goroutine is guaranteed to observe a write from another.

i. Happens-Before: transitive visibility relation of Sequenced-Before ∪ Synchronizes-With
ii. Within goroutine: observe program order (Sequenced-Before)
iii. Across goroutines: observed order may differ without synchronization
iv. Guarantee a read r of variable v observes write w if:
   - w happens before r
   - Other write to v happens before w or after r

Concurrency Patterns

Patterns:

i. Confinement:
   - Ad-hoc: only one goroutine mutates shared data
   - Lexical: restrict access to shared locations (e.g., expose only `<-chan T`, slice of array)
ii. Prevent goroutine leaks:
   - Creator ensures termination when work done, unrecoverable error, or when told to stop working
   - Beware nil channels: read/write blocks forever
iii. Error handling:
   - Return result + error along the same channel
   - Centralize decisions in a state-goroutine
iv. Pipeline:
   - Stages connected by channels; each stage is a group of goroutines running the same function
v. Fan-out / Fan-in:
   - Fan-out: replicate a slow stage across goroutines (independent work; order not guaranteed)
   - Fan-in: multiplex multiple input channels into one output channel
vi. Load balancing:
   - Pool of workers reads from a shared work channel
   - Each worker emits results; collect with fan-in

4. Classical Synchronisation Problems

Producer-Consumer

Processes share a bounded buffer of fixed size K, where producers add items until buffer full and consumers remove items when not empty.

Blocking Solution (init `not_full = K, not_emp = 0`):

```
// Producer
x = produce();

wait(not_full);
wait(mutex);
buffer.add(x);
signal(mutex);
signal(not_emp);
```

```
// Consumer
wait(not_emp);
wait(mutex);
x = buffer.get();
signal(mutex);
signal(not_full);

consume(item);
```

Sleeping Barber

Barber sleeps until a customer wakes him. If barber is busy and chairs are free, the customer sits or will leave if there are no chairs.

```
// Customer
wait(mutex);
if (customers==n){
  signal(mutex);
  exit();
}
customers += 1;
signal(mutex);

signal(customer);
wait(barber);
getHairCut();
signal(custDone);
wait(barbDone);

wait(mutex);
customers -= 1;
signal(mutex);
```

```
// Barber
while (1) {
  wait(customer);
  signal(barber);
  cutHair();
  wait(custDone);
  signal(barbDone);
}
```

Reader-Writer

Processes share a critical region, where readers can read simultaneously but writers must have exclusive access.

```
// Writer
wait(empty);
write();
signal(empty);
```

```
// Reader
wait(mutex);
readers++;
if (readers == 1)
  wait(empty);
signal(mutex);

read();

wait(mutex);
readers--;
if (readers == 0)
  signal(empty);
signal(mutex);
```

- Writer Starvation: possible if readers keep entering and `empty` is never signalled. Solutions:
  - Queue: all pass through `wait(queue)`
  - Writer-Priority: readers entering must `wait(readTry)`; but first waiting writer blocks new readers with `wait(readTry)` and does `signal(readTry)` when done

Barrier

```
int i_am_last = 0;
wait(mutex);
count++;
if (count == N) {
  i_am_last = 1;
  signal(barrier);
}
signal(mutex);

wait(barrier);
if (!i_am_last)
  signal(barrier);
```

Dining Philosophers

N philosophers around a circular table, with a single chopstick between. 2 chopsticks are needed to eat.

```
// Philosopher
void p(int i) {
  while (1) {
    // think
    take(i);
    eat();
    put(i);
  }
}

void safe_to_eat
   (int i) {
  if (state[i] ==
   HUNGRY &&
   state[LEFT] !=
   EATING &&
   state[RIGHT] !=
   EATING) {
    state[i] =
   EATING;
    signal(s[i]);
  }
}
```

```
// Take Chopstick
void take(int i) {
  wait(mutex);
  state[i] =
   HUNGRY;
  safe_to_eat(i);
  signal(mutex);
  wait(s[i]);
}

// Put Chopstick
void put(int i) {
  wait(mutex);
  state[i] =
   THINKING;

   safe_to_eat(LEFT);

   safe_to_eat(RIGHT)
  signal(mutex);
}
```

Limited Eater Solution, when at most $N-1$ philosophers eat concurrently, it's guaranteed at least one can eat:

```
void philosopher(int i) {
  while (1) {
    // think
    wait(seats);
    wait(chopstick[LEFT]);
    wait(chopstick[RIGHT]);
    eat();
    signal(chopstick[RIGHT]);
    signal(chopstick[LEFT]);
    signal(seats);
  }
}
```

5. Rust

Ownership and Borrowing

Rust enforces memory-safety at compile-time without GC, preventing data races and enabling concurrency.

i. Each value has exactly one owner
+ Ownership prevents double-free; owner frees
+ Borrowing prevents use-after-free (UAF)
+ Segfault-free

Operations:

i. Move: `T`
   - Transfers ownership; old binding cannot be used
   - Deep Copy: `clone()` is explicit (expensive)
ii. Shared Borrow: `&T`
   - Both bindings are read-only while shared
iii. Mutable Borrow: `&mut T`
   - Old binding is read-only while shared
   - New binding is mutable while shared

Threads

Operations (`std::thread`):

i. Spawn: `let h = spawn(move || {...})`
   - `move`: moves captured values into thread
ii. Join: `h.join()`
iii. Marker Traits:
   - `Copy`: safe to memcpy (i.e. `u32`, `f32`, no String)
   - `Send`: safe to transfer to another thread
   - `Sync`: safe to share reference between threads; `T` is `Sync` $\iff$ `&T` is `Send`
iv. Shared Ownership:
   - `Rc<T>`: within a thread
   - `Arc<T>`: across threads; has `Send` trait

Mutexes

Operations (`std::sync::Mutex`):

i. Construct: `let m = Mutex::new(x)`
ii. Lock-Guard: `let mut g Guard<T> = m.lock()`
   - Guard controls access: `*g += 1`
   - When guard goes out of scope, mutex unlocks

Atomics

Atomics are guaranteed lock-free indivisible types.

Operations (`std::sync::atomic`):

i. Construct: `let a = AtomicUsize::new(v)`
   - `AtomicBool`, `AtomicIsize`, `AtomicI16`
ii. Load: `a.load(SeqCst)`
iii. Store: `a.store(v, SeqCst)`
iv. Read-Modify-Write: `a.fetch_add(1, SeqCst)`, `a.swap(v, SeqCst)`

Multi-Producer, Single Consumer (MPSC)

MPSC provides a FIFO communication queue.

Operations (`std::sync::mpsc`):

i. Create: `let (tx, rx) = channel()`
ii. Clone Sender: `let tx2 = tx.clone()`
iii. Send: `tx.send(v)`
iv. Receive: `rx.recv()`

crossbeam Package

Crossbeam extends beyond `std`.

Operations:

i. Scoped Threads: `crossbeam::scope(...)`
   - Threads within borrow only from the scope
   - All spawned threads must end before scope exits
ii. MPMC Channels: `crossbeam::channel()`
iii. Bounded Channels: `crossbeam::bounded(C)`
iv. Backoff: `crossbeam::utils::Backoff::new()`
   - Exponentially slow retry reduces contention, spin

```
let mut v = vec![1, 2, 3, 4];
thread::scope(|s| {
  let (a, b) = v.split_at_mut(2); // disjoint
  s.spawn(|_| { for x in a { *x += 1; } });
  s.spawn(|_| { for x in b { *x += 1; } });
}).unwrap();
println!("{:?}", v); // safe: both returned
```

rayon Package

Rayon provides data-parallelism operations.

Operations (`rayon::prelude::*`):

i. Iterators:
   - Read-Only: `xs.par_iter()`
   - Mutable: `xs.par_iter_mut()`
ii. Combinators:
   - `map(...)`, `filter(...)`, `for_each(...)`
   - Aggregation: `reduce(...)`/ `sum()`
iii. Sort: `xs.par_sort()`/ `xs.par_sort_unstable()`

```
let max = AtomicI64::new(MIN);
vec.par_iter().for_each(|n| {
  loop {
    let old = max.load(SeqCst);
    if *n <= old { break; }
    let r = max.compare_and_swap(old, *n, SeqCst);
    if r == old {
      // swapped
      break;
    }
  }
})
```

Non-blocking I/O

Non-blocking I/O enables concurrency with one thread:

i. Blocking I/O: thread sleeps until data arrives
ii. `epoll`: kernel reports ready file descriptors
iii. Event Loop: wait for ready fds → handle I/O

Futures

Futures represent work that will produce a value later:

i. `Future`: trait driven by `poll(Context)`:
   - Runs until it can no longer make progress
   - Returns `Poll::Ready(T)` or `Poll::Pending`
ii. `Context` provides `wake()` (notify executor to repoll)
iii. Combinators: `.then()`

Executors are user-space schedulers that drive futures:

i. `Pending` ⇒ future arranges a `wake()` when it can progress again
ii. If nothing can progress: sleep until a `wake()`
iii. Threading with Tokio:
   - Single-thread: one OS thread polls all futures
   - Multi-thread: futures may run on multiple cores
   - Shared data still needs synchronization

Async/Await

Async/Await are syntactic sugar for composing Futures.

i. `async fn` returns a `Future`
ii. `.await` waits for a future (only inside `async`)

+ Zero-cost Abstraction: compiler lowers `async fn` into a state machine implementing `poll()`
+ Lower memory use with stackless coroutines
+ Lower scheduling overhead from context switches
+ Scales to very high concurrency (esp. I/O bound)
– Block inside async sleeps the executor thread
– Cooperative: tasks yield only at `.await`
– CPU-heavy work can starve other tasks
– No recursion; future size fixed at compile time

6. Testing

Bug Types:

i. Unwanted Blocking:
   - Deadlock/Livelock: no progress, operations are not "completed", effects do not become visible
   - IO: waiting for external input
ii. Race Conditions:
   - Data races ⇒ undefined behavior (unsynchronized concurrent access)
   - Broken invariants:
     – Double-free
     – Dangling pointers $\implies$ UAF
     – Memory corruption from read partial update
   - Lifetime issues: thread outlives data it accesses

Techniques:

i. Review the code:
   - Identify shared data and how it is protected
   - Track held mutexes and find lock ordering issues
   - Check cross-thread ordering enforcement
   - Validate lifetimes (e.g. `join()` on exceptions)
ii. Testing:
   - Run the smallest amount of code that can demonstrate the problem
   - Remove concurrency to confirm it is concurrency-related
   - Run on multicore and (when useful) single-core

Dynamic Instrumentation

Valgrind:

i. Shadows registers and memory to validate memory operations:
   each allocation freed once, each access to allocated space, each reach from initialized memory
ii. High overhead: `10-20x`
iii. Usage: `valgrind --tool=memcheck <prog ...>`

Helgrind:

i. Intercepts `pthread` operations to validate thread operations:
   miuse of POSIX threads API, potential deadlocks from lock-order cycles, data races from happens-before analysis
ii. Very high overhead: `~100x`
iii. Usage: `valgrind --tool=helgrind <prog ...>`

Compiler Instrumentation

Sanitizers:

i. Compile-time instrumentation; supported by `clang` and `gcc`
ii. Lower overhead: `~5-10x`
iii. Instruments memory accesses / function boundaries and tracks memory state

AddressSanitizer (ASan):

i. Detects addressability issues
ii. Typical cost: `~2x` slowdown, `1.5-3x` memory overhead
iii. Usage: `-fsanitize=address`

ThreadSanitizer (TSan):

i. Detects data races using happens-before tracking
ii. Also reports thread / mutex / lock-order issues
iii. Typical cost: `5-15x` slowdown, `5-10x` memory overhead
iv. Usage: `-fsanitize=thread -fPIE -pie`

Model Checking

Check Formal Specifications:

i. Build a model in a special DSL / new language
ii. Check: constraints, unexpected behaviour, deadlock
iii. Why spend time:
   - Check designs before costly implementation
   - Prove properties for existing code
   - Enable aggressive optimizations without breaking correctness

\+ Rigorous, check all traces exhaustively
\+ Produces trace violating requirement
− Tedious to define correct specification

Approaches:

i. Write a formal specification and check it (with model checker/ proof assistant)
ii. Use specification to write minimal code:
iii. Add invariants as comments to assist checker:

Model Checkers:

i. TLA+ (TLC): temporal properties; good for concurrent/distributed systems
   - Write a TLA+ specification; use TLC to exhaustively explore the state space
   - Check safety (invariants) and liveness (temporal)
   - Reports errors with a counterexample trace
   - Model Structure:
     `Init` (initial states) + `Next` (state transitions)
   - Uses fairness to rule out infinite "stuttering" (non-progress) executions
ii. Coq: interactive proofs; generate code (OCaml/Haskell/Scheme)
iii. Alloy: relational logic; good for modeling structures

Challenges of Distributed Systems:

i. Reliability: in case of failures
ii. No global clock or ordering: need memory model
iii. Consistency
iv. Consensus